

PHP-Analyse: clicksolutions/contao-login-screen

Stand 08.10.2026 · Skills: `php-pro`, `php-best-practices` · PHP-Zielversion: `^8.3` (Runtime 8.5.11) · Contao `^5.7`

1. Zusammenfassung

Das Bundle ist klein (380 PHP-Zeilen, davon 291 im Hook-Listener), sauber getypt und defensiv gebaut. Alle Dateien nutzen `declare(strict_types=1)`, PHPStan Level 9 läuft fehlerfrei durch, alle Dateien sind syntaktisch korrekt, es gibt keine SQL-, XSS- oder Upload-Angriffsfläche im eigenen Code. Offene Punkte betreffen vor allem fehlende Tests, die regexbasierte HTML-Manipulation, Contao-idiomatische Konfiguration und PHP-8.3-Feinschliff.

Prüfung	Ergebnis
PHPStan Level 9 (<code>phpstan analyse</code>)	0 Fehler
<code>php -l</code> auf alle PHP-Dateien	ohne Fehler
PHP-CS-Fixer <code>@PSR12</code> (Dry-Run, <code>src/</code>)	1 Abweichung (leerer Konstruktor-Body <code>{}</code>)
Unit-Tests / PHPUnit	nicht vorhanden

Priorität	Anzahl
Mittel	4
Niedrig	8
Hinweis	3

2. Geprüfte Dateien

Datei	Zeilen	Bewertung
<code>src/EventListener/Hooks/OutputBackendTemplateListener.php</code>	291	Kernlogik; solide, siehe Findings
<code>src/ClickSolutionsContaoLoginScreenBundle.php</code>	25	ok
<code>src/ContaoManager/Plugin.php</code>	35	ok
<code>src/DependencyInjection/...Extension.php</code>	29	ok
<code>contao/dca/tl_page.php</code>	40	funktioniert; Verbesserungen möglich
<code>config/services.yaml</code> , <code>composer.json</code> , <code>phpstan.neon</code>	–	siehe Findings

3. Findings

Mittel

M1 – [test] Keine automatisierten Tests

gesamtes Bundle, `composer.json` php-pro verlangt Tests und eine Abdeckung $\geq 80\%$. Der Listener besteht überwiegend aus reiner String-Logik (`prepareBody`, `insertBefore`, `replaceOnce`, `moveFrontendLinkIntoMain`) und ist deshalb sehr gut testbar. Gerade bei Änderungen am Core-Markup von `be_login` würde ein Test mit einem Markup-Fixture Regressionen sofort zeigen (Hinweis in CLAUDE.md: ein altes Template-Override hat bereits Passkey-/SSO-Login gebrochen).

Empfehlung: `phpunit/phpunit` als `require-dev`, Tests mit Fixture-HTML für `be_login` und `be_login_two_factor`; Studio/Framework als Mocks.

M2 – [solid/robustness] HTML-Manipulation per Regex und `strrpos`

`OutputBackendTemplateListener.php:89, 99, 118, 135, 165` Die Injektion hängt an festen Ankern (`<body`, `<h1`, `</main>`, `</head>`, `<p class="fe-link">`). Das ist bewusst gewählt und fehlende Anker werden geloggt – mit einer Ausnahme (M3). Der Ansatz ist aber anfällig bei Markup-Änderungen (z. B. zweite CSS-Klasse am `fe-link`-Absatz, Kommentare mit `</main>`).

Empfehlung: Anker-Logik durch Tests absichern (M1); alternativ `Dom\HTMLDocument` (PHP 8.4+) erst nutzen, wenn die Mindestversion angehoben wird. Die Verwendung von `strrpos` für `</main>` ist korrekt, sollte aber im Kommentar begründet sein.

M3 – [error] Stilles Überspringen beim `fe-link`

`OutputBackendTemplateListener.php:118–120` Passt das Regex nicht, wird ohne Log zurückgegeben – das ist beim Zwei-Faktor-Template gewollt, bei `be_login` jedoch ein stiller Fehlerfall (Core ändert die Klasse → Link liegt nach der Anpassung unlesbar über dem Hintergrund).

Empfehlung: nur für `be_login` warnen (Template-Name an die Methode durchreichen).

M4 – [config] Palette per String-Konkatenation statt `PaletteManipulator`

`contao/dca/tl_page.php:12–13` `.=` auf `$GLOBALS['TL_DCA']['tl_page']['palettes']` [...] erzeugt bei geänderter/entfernter Palette einen Notice und hängt immer ans Ende. Contao-Standard ist `Contao\CoreBundle\DataContainer\PaletteManipulator` (`addLegend()` / `addField()`), das auch Position und fehlende Paletten sicher behandelt.

Niedrig

N1 – [modern] Typisierte Klassenkonstanten (PHP 8.3)

`OutputBackendTemplateListener.php:30–32` `private const string ASSET_PACKAGE = ... / private const array TEMPLATES = ...`. Passt zu `php: ^8.3`.

N2 – [modern] `#[\Override]` statt `{@inheritdoc}`

`Plugin.php:23–25`, `Extension.php:21–23`, `Bundle.php:22` In PHP 8.3 zeigt `#[\Override]`

Tipfehler/Signaturänderungen zur Compile-Zeit an; die redundanten `{@inheritdoc}` - und „Class X.“-Docblocks können entfallen.

N3 – [solid] Klassen nicht `final`

alle vier Klassen in `src/` Listener, Plugin, Extension und Bundle sind nicht zur Vererbung gedacht. `final` macht das explizit (php-pro: „final/readonly where applicable“). Beim Listener wird Mocking in Tests über Interfaces/Dependencies statt Subklassen gelöst.

N4 – [modern] Hook per Attribut und Autowiring

`config/services.yaml`, `OutputBackendTemplateListener.php:28` Statt manueller Argumentliste und `contao.hook`-Tag genügt `#[AsHook('outputBackendTemplate')]` (`Contao\CoreBundle\DependencyInjection\Attribute\AsHook`) plus `autowire/autoconfigure;` `$projectDir` via `#[Autowire('%kernel.project_dir%')]`. Reduziert Konfigurationsdrift bei Konstruktoränderungen und entspricht dem Muster der anderen Clicksolutions-Bundles (Autodiscovery, Attribute).

N5 – [solid] Veränderlicher Zustand im Service

`OutputBackendTemplateListener.php:37, 59, 177–181` `$warned` ist Instanzzustand, der pro Aufruf zurückgesetzt wird. Funktioniert, widerspricht aber der Idee zustandsloser Services (relevant z. B. bei Worker-Runtimes). Alternative: Warnungen lokal sammeln oder Duplikat-Logs akzeptieren (eine Anfrage = ein Aufruf).

N6 – [perf] Bild wird vor Anker-Prüfung generiert

`OutputBackendTemplateListener.php:85–92, 95–103` Hintergrund und Logo werden gerendert (Studio/Resize-Konfiguration), bevor feststeht, dass der Anker existiert. Bei fehlendem Anker entsteht unnötiger Aufwand; Reihenfolge umdrehen (Anker prüfen, dann rendern) oder lazy über Callback – der `replaceOnce`-Callback tut das für den Logo-Pfad teilweise nicht, da `$logoHtml` vorher berechnet wird.

N7 – [config] Inkonsistente Dateieindungen

`contao/dca/tl_page.php:18` vs. `:24` Logo: `jpg,svg,png`; Hintergrund: `jpg,jpeg,png,webp,avif`. `jpeg/webp` fehlen beim Logo (und `svg` beim Hintergrund ist bewusst ausgeschlossen). Prüfen, ob gewollt, ggf. angleichen.

N8 – [psr] Stil-Kleinigkeiten

Listener:46 · Extension.php:25 · `tl_page.php:20–34` (a) `{}` beim Konstruktor weicht vom PSR-12-Fixer ab (gültig nach PER-CS 2.0; `.php-cs-fixer`-Config mit `@PER-CS` würde das festschreiben). (b) `__DIR__.'../../config'` ohne Leerzeichen, im Listener dagegen `$a . $b`. (c) SQL-Definitionen mit doppelten Anführungszeichen ohne Interpolation, fehlende abschließende Kommas. (d) Mehrere über 150 Zeichen lange Zeilen (Listener:82, 90, 108).

Hinweise

H1 – [error] Breites `catch (\Throwable)` ist hier gerechtfertigt

Listener:62 Regel `error-try-catch-specific` verlangt spezifische Exceptions; hier ist der Fail-Open-Ansatz (Login darf nie brechen, Fehler werden mit Exception geloggt) richtig und dokumentiert. Kein Handlungsbedarf.

H2 – [composer] Transitive Abhängigkeiten

composer.json:11–14 Genutzt werden `symfony/asset`, `symfony/http-foundation`, `psr/log` und `contao/image`, aber nur `contao/core-bundle` ist deklariert. Für Contao-Bundles üblich und über den Core abgedeckt. Fehlende `homepage` / `support` -Felder und ein `cs` -Script wären Komfort. Der `classmap` -Eintrag für `contao/` hat nach dem Entfernen von `config.php`, Sprachdatei und Template keinen Inhalt mehr (nur noch `dca/`, ausgeschlossen) und kann samt `exclude-from-classmap` entfallen; CLAUDE.md erwähnt ihn noch.

H3 – [a11y] Alternativtext des Hintergrundbildes

Listener:248–255 Das Hintergrundbild ist dekorativ. Je nach Meta-Daten der Datei kann das `Legacy-image` -Template einen Alt-Text ausgeben, der von Screenreadern vorgelesen wird. Prüfen und ggf. per `setMetadata / alt=""` neutralisieren.

4. Positiv

- Strikte Typisierung durchgängig, `readonly` -Constructor-Promotion, moderne `null|string` -Unions, `mixed` nur an Eingangsgrenzen (`findFile`, `generateBackgroundImageHtml`) mit sofortiger Validierung.
- Ausgabe-Escaping korrekt: Text wird mit `strip_tags` + `htmlspecialchars` ausgegeben, URL des Assets escaped, Klassen aus Konstanten.
- Kein globaler Zustand und keine statischen Contao-Aufrufe: `ContaoFramework` -Adapter, `RequestStack`, Studio per DI.
- Fail-open-Design mit Logging; Warn-Deduplizierung; Prüfung auf lesbare Dateien vor dem Rendern.
- Kein Template-Override mehr (Kompatibilität mit Core-Login/Passkey), CSS nur auf Login-Seiten geladen und mit `body.cs_cls_login` gescoped.
- Übersetzungen als Symfony-Translations; Namensschema `cs_cls_*` konsistent.

5. Empfohlene Reihenfolge

1. PHPUnit + Fixture-Tests für den Listener (M1), dabei M3 beheben.
2. `PaletteManipulator` in der DCA (M4).
3. PHP-8.3-Feinschliff gesammelt: typisierte Konstanten, `#[\Override]`, `final`, `#[AsHook]` (N1–N4).
4. Housekeeping: classmap entfernen, Dateiendungen angleichen, PER-CS-Config (H2, N7, N8).